



**TITULO:**

**INSTITUTO TECNOLOGICO SUPERIOR SUDAMERICANO**

**AUTOR(ES):** *Pablo Ortega*

**MATERIA:** Programacion Orientada Objetos

**DOCENTE:** Carolina Aldas

**CURSO:** 2DO

**CARRERA:** "DESARROLLO DE SOFTWARE"

**TEMA:** Backend con NestJS, PostgreSQL y TypeORM

**FECHA DE ELABORACIÓN:** Jueves, 6 de noviembre de 2025, 00:00

**FECHA DE ENTREGA:** Miércoles, 12 de noviembre de 2025, 23:59

## 1. Crear el Proyecto:

Primero entramos en nuestro cmd y creamos el proyecto mkdir

```
Microsoft Windows [Versión 10.0.26200.7019]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\pablo>cd desktop

C:\Users\pablo\Desktop>cd objetospoo

C:\Users\pablo\Desktop\objetospoo>cd sem5

C:\Users\pablo\Desktop\objetospoo\sem5>cd notification-api

C:\Users\pablo\Desktop\objetospoo\sem5\notification-api>
```

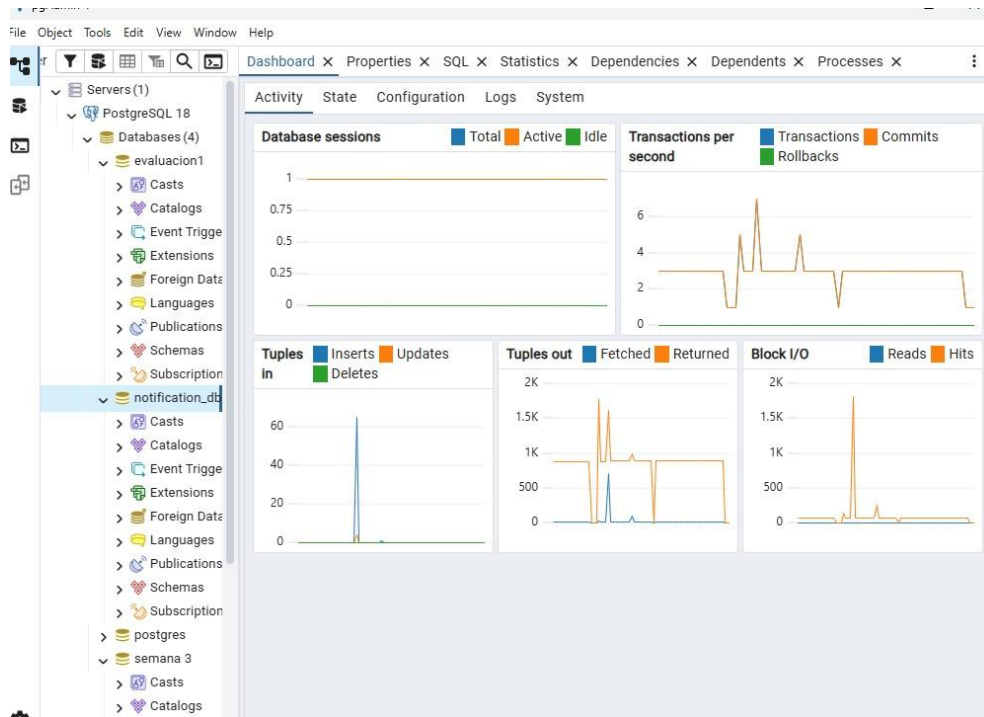
En este caso yo lo tengo creado ya pero lo creamos con nest new notification-api

## 2. Instalar dependencias necesarias

Instalamos la dependencias necesarias con este comando: `npm install @nestjs/typeorm typeorm pg` en nuestro cmd

En este caso no muestro imagen adjunta por que ya lo tengo creado pero en todo caso ese seria el comando en nuestro cmd

## 3. Configuramos la conexión a la base de datos



#### 4. Creamos Módulo, Servicio y Controlador de Notificaciones

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER 28

Página de códigos activa: 65001

C:\Users\HP\Desktop\ObjetosPOO\sem5\notification-api>nest g module notifications
CREATE src/notifications/notifications.module.ts (94 bytes)
UPDATE src/app.module.ts (354 bytes)

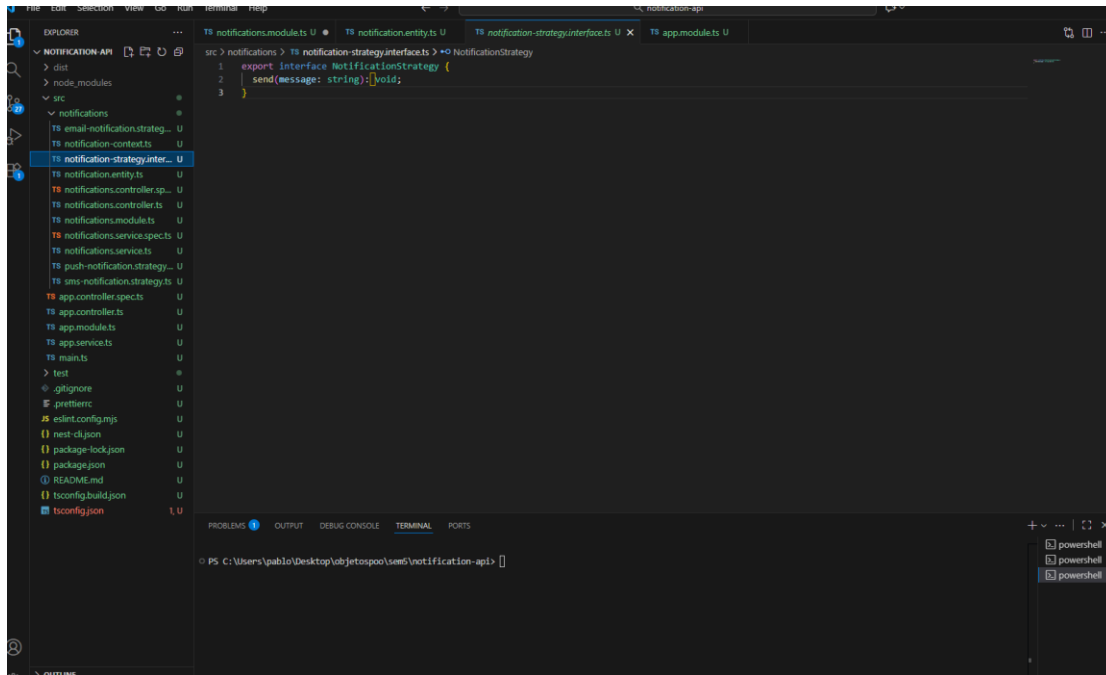
C:\Users\HP\Desktop\ObjetosPOO\sem5\notification-api>nest g service notifications
CREATE src/notifications/notifications.service.ts (101 bytes)
CREATE src/notifications/notifications.service.spec.ts (527 bytes)
UPDATE src/notifications/notifications.module.ts (195 bytes)

C:\Users\HP\Desktop\ObjetosPOO\sem5\notification-api>nest g controller notifications
CREATE src/notifications/notifications.controller.ts (119 bytes)
```

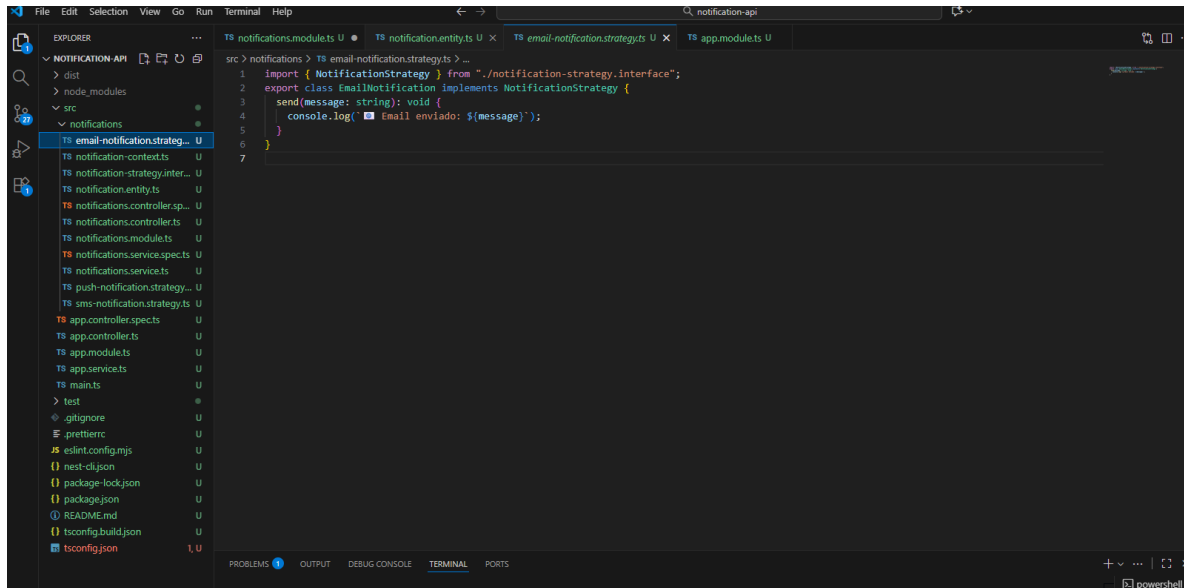
#### 5. Crear la Entidad de Notificación

```
File Edit Selection View Go Run Terminal Help
src/notifications > TS notification.entity.ts > TS Notification
1 import { Column, CreateDateColumn, Entity, PrimaryGeneratedColumn } from "typeorm";
2
3 @Entity()
4 export class Notification {
5   @PrimaryGeneratedColumn()
6   id: number;
7
8   @Column()
9   message: string;
10
11   @Column()
12   type: string;
13
14   @CreateDateColumn()
15   created_at: Date;
16 }
```

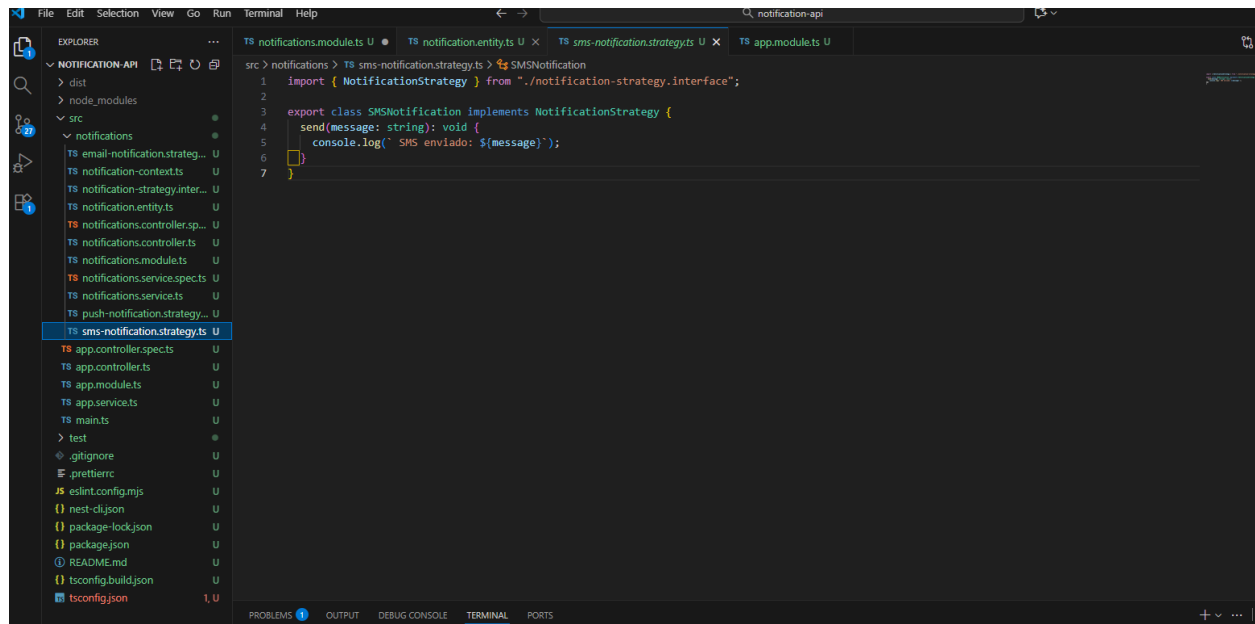
## 6. Crear la interfaz de Estrategia



## 7. Implementar Estrategias de Notificación creando dentro de la carpeta Notifications



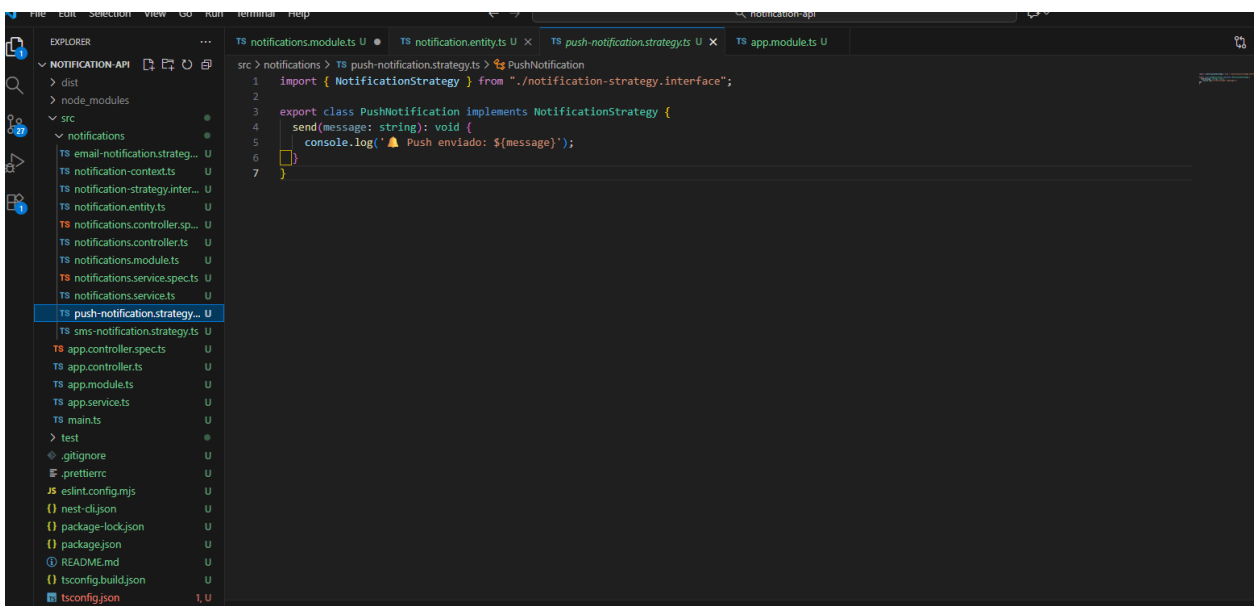
Luego creamos sms-notification.strategy.ts



The screenshot shows the Visual Studio Code editor with the Explorer sidebar on the left. The Explorer sidebar shows a project structure for 'NOTIFICATION-API' with a 'src' directory containing a 'notifications' subdirectory. The 'notifications' directory contains several files, including 'sms-notification.strategy.ts' which is currently selected and highlighted. The main editor area shows the code for 'sms-notification.strategy.ts'.

```
src > notifications > TS sms-notification.strategy.ts > SMSNotification
1 import { NotificationStrategy } from "../notification-strategy.interface";
2
3 export class SMSNotification implements NotificationStrategy {
4   send(message: string): void {
5     console.log(' SMS enviado: ${message}');
6   }
7 }
```

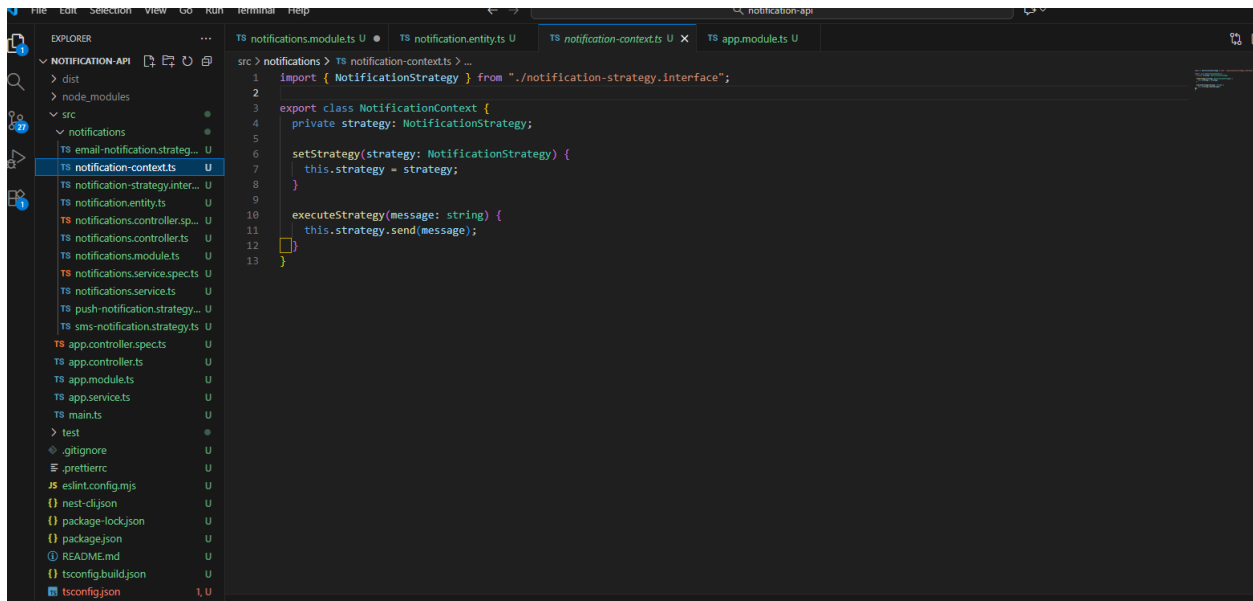
Luego creamos push-notification.strategy.ts



The screenshot shows the Visual Studio Code editor with the Explorer sidebar on the left. The Explorer sidebar shows a project structure for 'NOTIFICATION-API' with a 'src' directory containing a 'notifications' subdirectory. The 'notifications' directory contains several files, including 'push-notification.strategy.ts' which is currently selected and highlighted. The main editor area shows the code for 'push-notification.strategy.ts'.

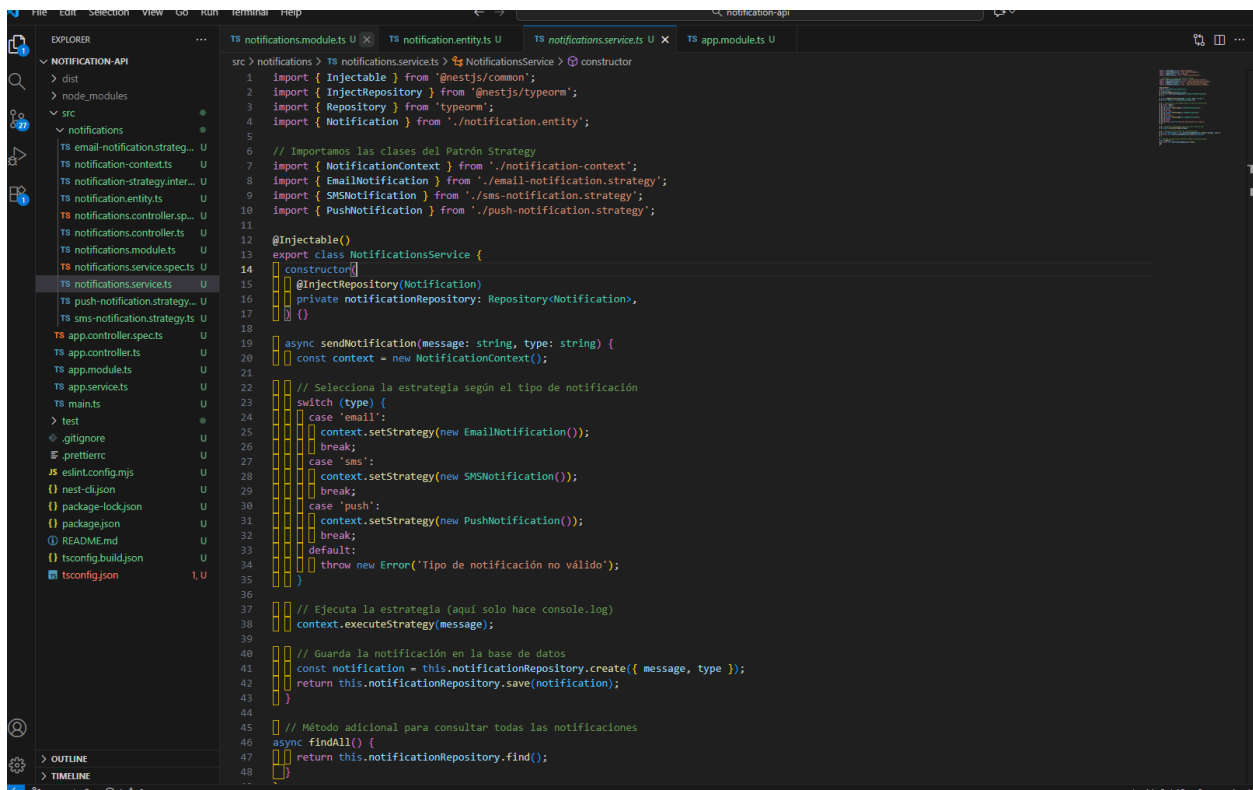
```
src > notifications > TS push-notification.strategy.ts > PushNotification
1 import { NotificationStrategy } from "../notification-strategy.interface";
2
3 export class PushNotification implements NotificationStrategy {
4   send(message: string): void {
5     console.log(' 📡 Push enviado: ${message}');
6   }
7 }
```

## 8. Crear la clase Contexto



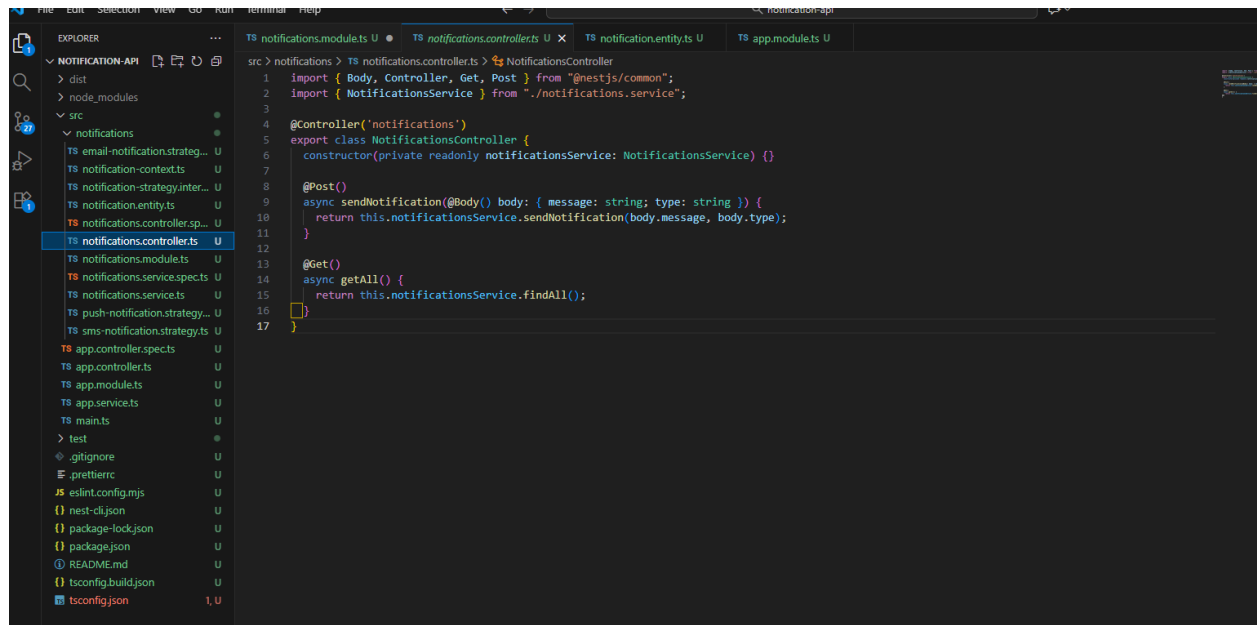
```
1 import { NotificationStrategy } from './notification-strategy.interface';
2
3 export class NotificationContext {
4   private strategy: NotificationStrategy;
5
6   setStrategy(strategy: NotificationStrategy) {
7     this.strategy = strategy;
8   }
9
10  executeStrategy(message: string) {
11    this.strategy.send(message);
12  }
13 }
```

## 9. Implementar nuestra lógica en el Servicio( service.ts)

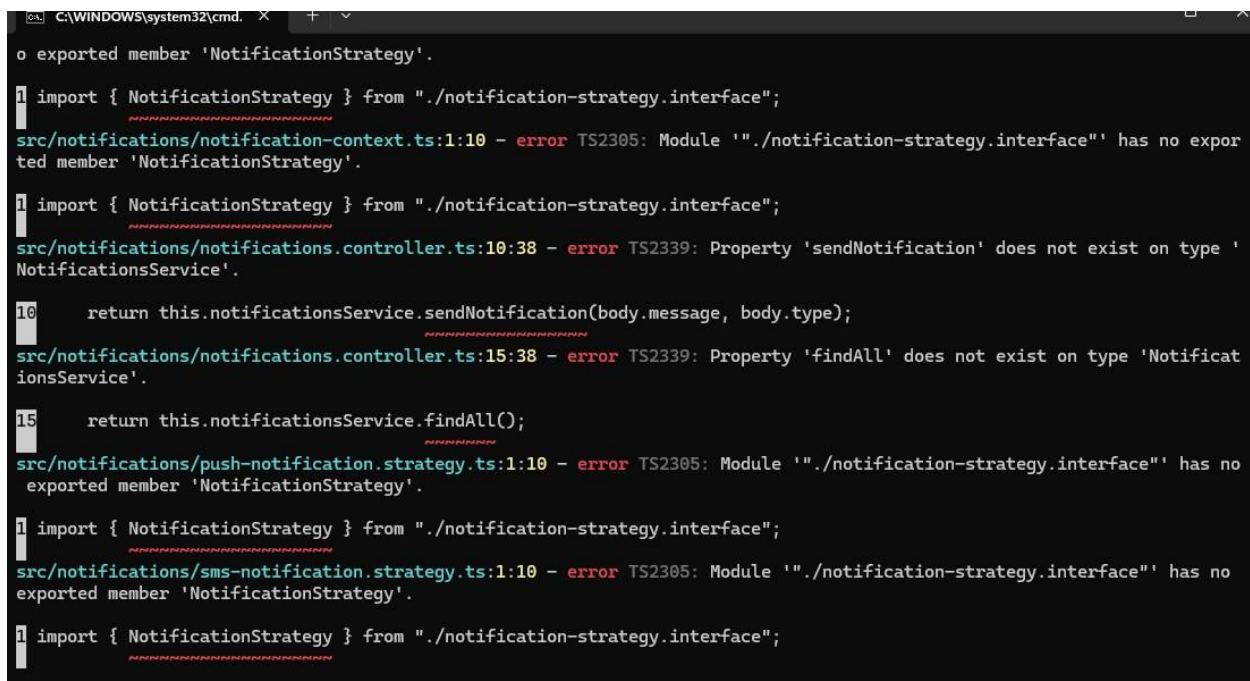


```
1 import { Injectable } from '@nestjs/common';
2 import { InjectRepository } from '@nestjs/typeorm';
3 import { Repository } from 'typeorm';
4 import { Notification } from './notification.entity';
5
6 // Importamos las clases del Patrón Strategy
7 import { NotificationContext } from './notification-context';
8 import { EmailNotification } from './email-notification.strategy';
9 import { SMSNotification } from './sms-notification.strategy';
10 import { PushNotification } from './push-notification.strategy';
11
12 @Injectable()
13 export class NotificationsService {
14   constructor(
15     @InjectRepository(Notification)
16     private notificationRepository: Repository<Notification>,
17   ) {}
18
19   async sendNotification(message: string, type: string) {
20     const context = new NotificationContext();
21
22     // Selecciona la estrategia según el tipo de notificación
23     switch (type) {
24       case 'email':
25         context.setStrategy(new EmailNotification());
26         break;
27       case 'sms':
28         context.setStrategy(new SMSNotification());
29         break;
30       case 'push':
31         context.setStrategy(new PushNotification());
32         break;
33       default:
34         throw new Error('Tipo de notificación no válido');
35     }
36
37     // Ejecuta la estrategia (aquí solo hace console.log)
38     context.executeStrategy(message);
39
40     // Guarda la notificación en la base de datos
41     const notification = this.notificationRepository.create({ message, type });
42     return this.notificationRepository.save(notification);
43   }
44
45   // Método adicional para consultar todas las notificaciones
46   async findAll() {
47     return this.notificationRepository.find();
48   }
49 }
```

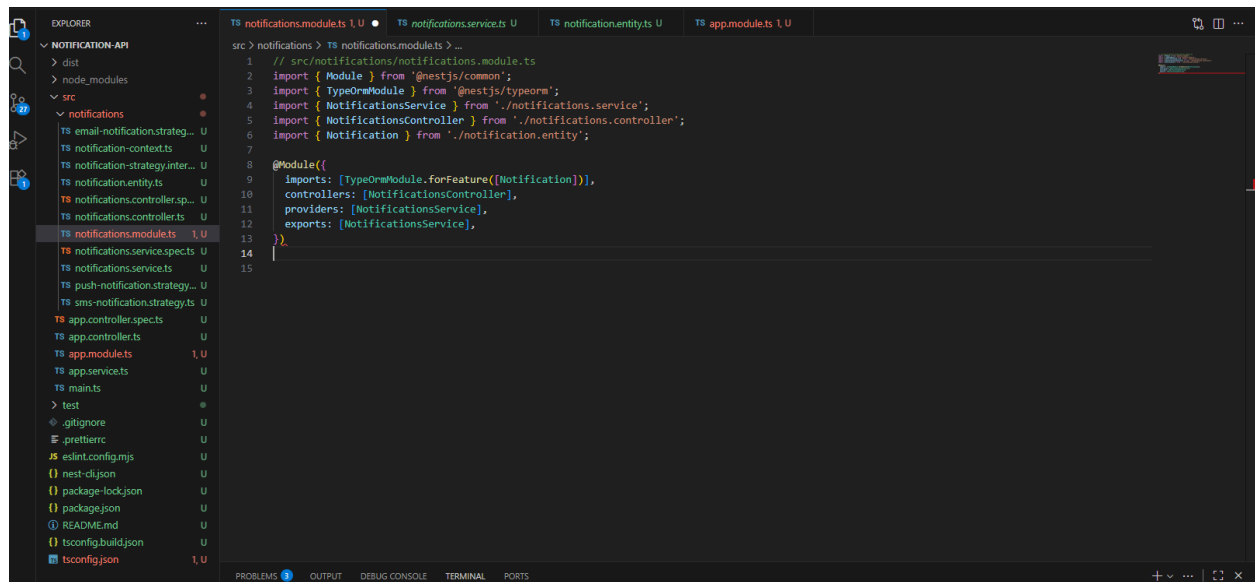
## 10. Implementar el Controlador



Levantamos nuestro proyecto con `npm run start` pero para ello debemos tener abierta nuestra base de datos en `psjadmini`

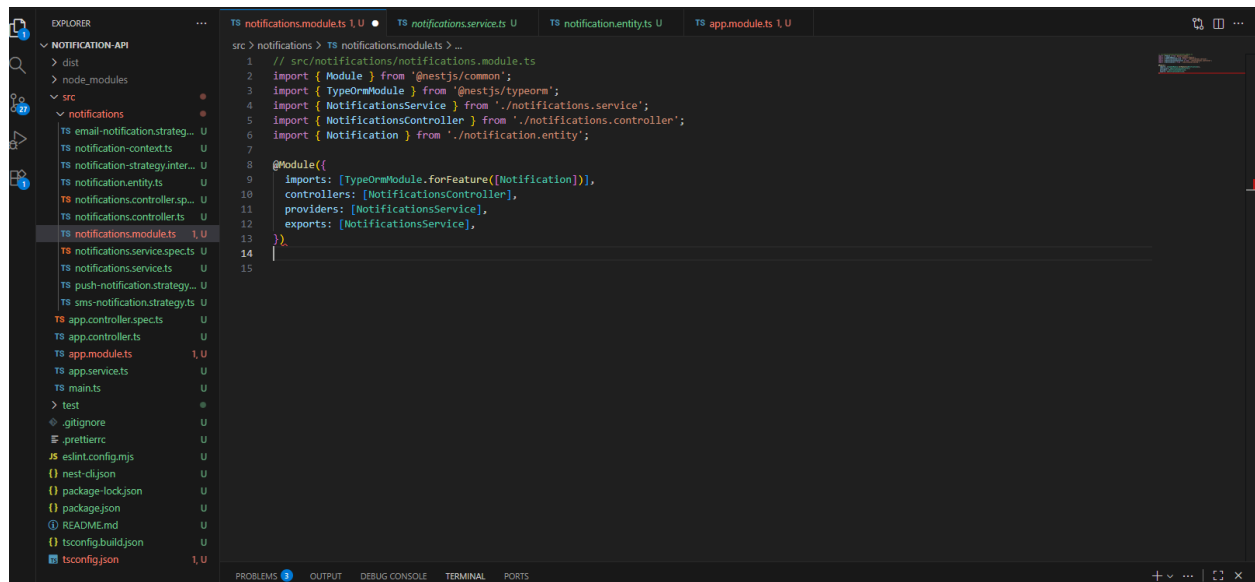


Ami al momento de levantar el proyecto me fue dando estos errores los fui corrigiendo con ayuda de la profesora en clases



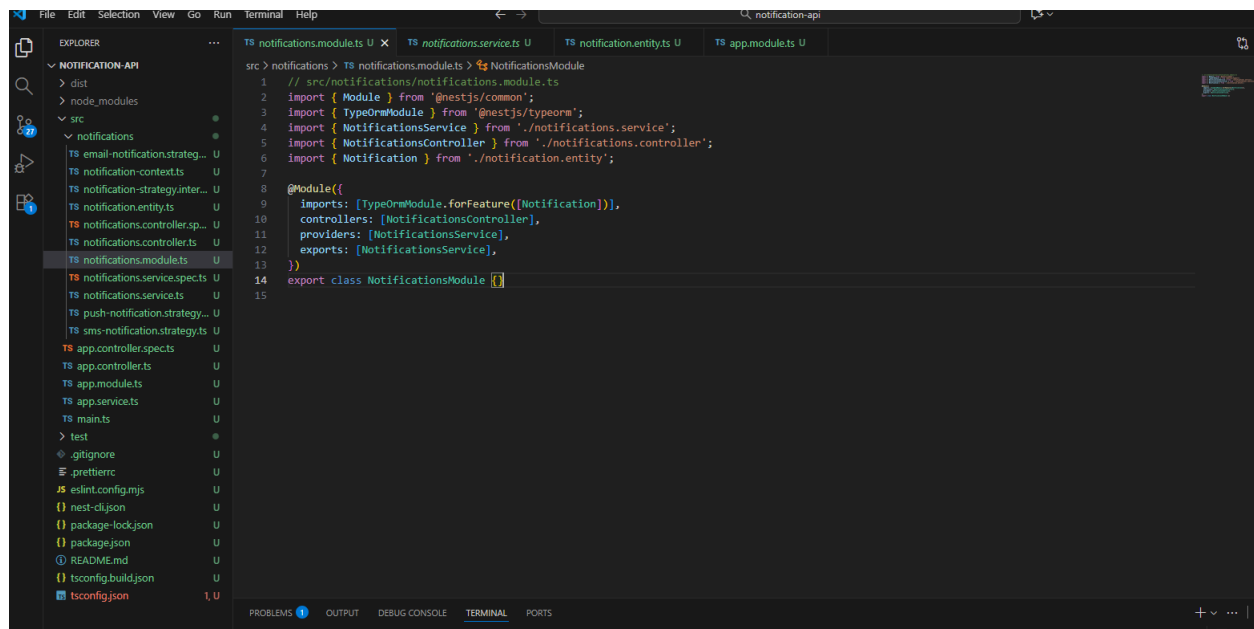
```
src > notifications > TS notifications.module.ts > ...
1 // src/notifications/notifications.module.ts
2 import { Module } from '@nestjs/common';
3 import { TypeOrmModule } from '@nestjs/typeorm';
4 import { NotificationsService } from './notifications.service';
5 import { NotificationsController } from './notifications.controller';
6 import { Notification } from './notification.entity';
7
8 @Module({
9   imports: [TypeOrmModule.forFeature([Notification])],
10  controllers: [NotificationsController],
11  providers: [NotificationsService],
12  exports: [NotificationsService],
13 })
14
15
```

Y también me daba error en el notification.module.ts

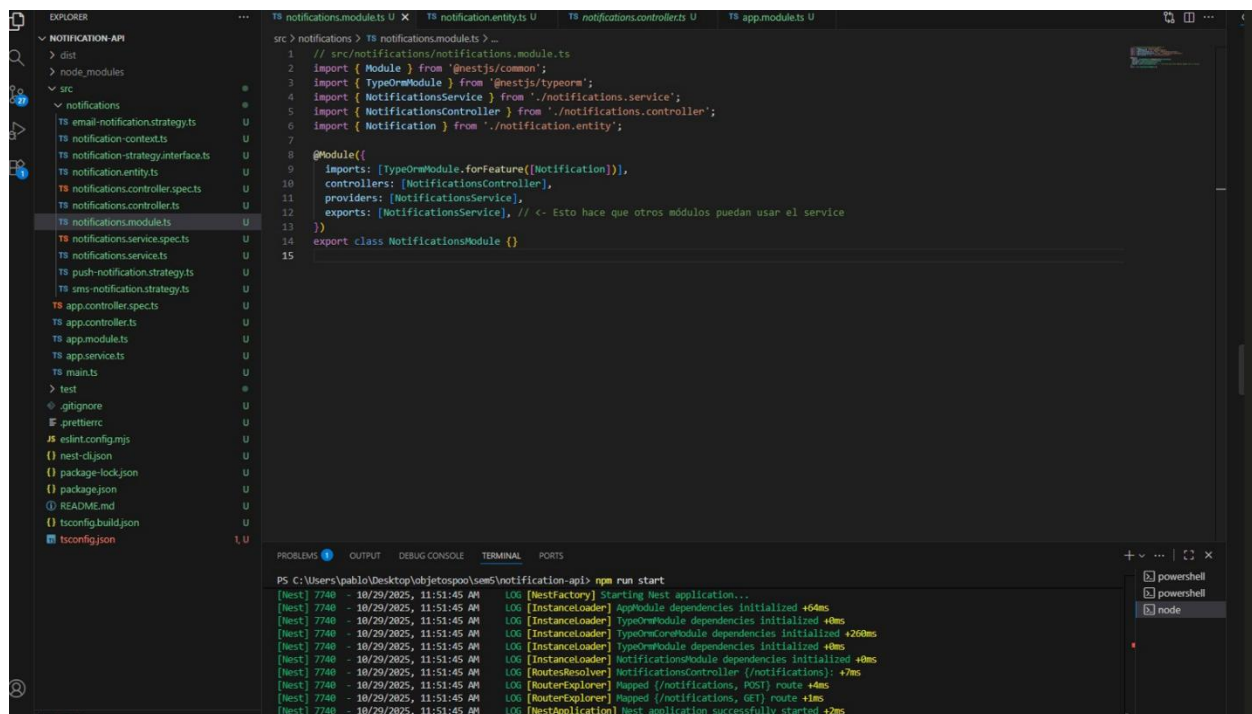


```
src > notifications > TS notifications.module.ts > ...
1 // src/notifications/notifications.module.ts
2 import { Module } from '@nestjs/common';
3 import { TypeOrmModule } from '@nestjs/typeorm';
4 import { NotificationsService } from './notifications.service';
5 import { NotificationsController } from './notifications.controller';
6 import { Notification } from './notification.entity';
7
8 @Module({
9   imports: [TypeOrmModule.forFeature([Notification])],
10  controllers: [NotificationsController],
11  providers: [NotificationsService],
12  exports: [NotificationsService],
13 })
14
15
```

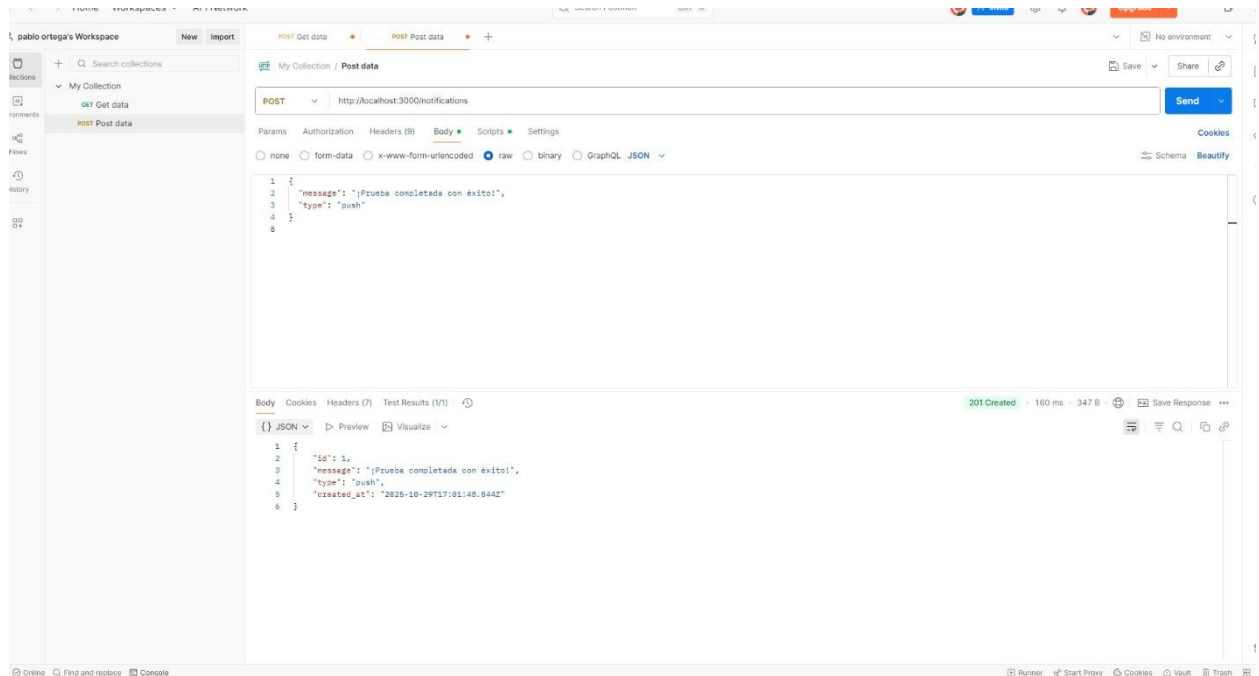
Pero lo corregui exportando el `export class NotificationsModule {}`



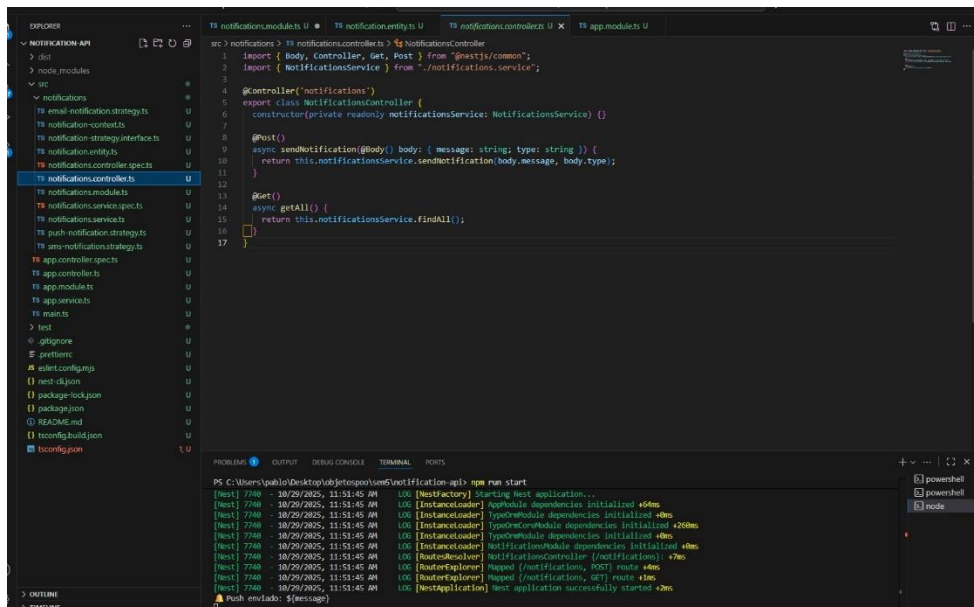
Y ahora si levantamos el proyecto con npm run start



11. Crea dos endpoints y aseguramos en POST `http://localhost:3000/notifications`, finalmente sale todo correctamente



Como vemos sale el mensaje



Y también aca